

COLETÂNEA NEXUS AFFILIA'ITE • VOL. 02

Lovable

Engenharia de Aplicações Full-Stack com IA
Conversacional



Por MMN AI-to-AI · Coleção Técnica Nexus

MMN AI-to-AI · 2026 · v1.0 · 28 páginas

Sobre este ebook

Lovable não é "mais um construtor de sites por IA". É o primeiro **agente de engenharia full-stack conversacional** que entende requisitos de produto, projeta arquitetura, escreve código React + TypeScript + Supabase, executa migrations, faz deploy na Vercel e mantém a aplicação em produção — tudo a partir de uma conversa em português.

Para o ecossistema Nexus, isso significa: um afiliado com uma ideia de SaaS consegue ir do zero ao produto validado em **72 horas**, sem contratar desenvolvedor, designer ou DevOps. O volume 02 da coletânea entrega o manual técnico para fazer isso com qualidade de engenharia, não com gambiarra de demo.

O que você vai dominar:

- A arquitetura interna do Lovable: Claude Sonnet 4 + tools de execução + RAG sobre docs
- Como escrever prompts de produto que viram PRD → arquitetura → código
- Stack técnica gerada (React 18, Vite, Tailwind, Supabase, Stripe, Resend)
- Patterns de prompts para multi-tenancy, autenticação, RBAC e billing
- Debugging, governança, versionamento e estratégia de evolução

"Lovable não substitui o desenvolvedor. Substitui a fricção entre a ideia e o software rodando." — Manifesto Nexus Engineers

| Sumário

1. A nova era do software conversacional	04
2. Arquitetura técnica do Lovable	06
3. Do prompt ao PRD executável	09
4. Stack automática e customização	12
5. Multi-tenancy, auth e RBAC	15
6. Billing, Stripe e SaaS recorrente	18
7. Integrações com APIs externas	21
8. Debugging, logs e observabilidade	24
9. Governança, segurança e escala	26
10. Glossário técnico e exercícios práticos	28

► Mapa: da frase em português à aplicação deployada em produção.

1 A nova era do software conversacional

Entre 2023 e 2026, três ondas redefiniram quem pode criar software. A primeira foi o **vibe coding** com ChatGPT: prompts isolados geravam snippets de código que developers colavam em suas IDEs. A segunda foi o **GitHub Copilot Workspace**: a IA passou a entender o repositório inteiro, mas ainda exigia um humano orquestrando cada PR. A terceira — onde estamos com o Lovable — é a **aplicação agentic**: a IA é o engenheiro, ela planeja, escreve, testa, faz deploy e monitora.

1.1 — O que muda para o afiliado Nexus

Historicamente, um afiliado que quisesse lançar um SaaS próprio (curso plataforma, ferramenta de IA, comunidade) precisava levantar R\$ 30k-150k e 3-9 meses. Com Lovable, esse CAPEX cai para R\$ 0-3k e 3-9 dias. Isso não é "democratização" — é **destruição criativa de barreiras de entrada**.

1.2 — O novo perfil: operador de produto + IA

Os profissionais mais bem pagos de 2026 não são mais "devs full-stack" puros. São **PM-Engineers**: pessoas que entendem o domínio do problema, sabem especificar requisitos com precisão e operam a IA como se fosse um time de devs juniores infinitamente escalável.

Armadilha comum: tratar Lovable como "magic box" e pedir coisas vagas. O resultado é um app que parece bom no print mas quebra em produção. Este ebook é o antídoto.

1.3 — Casos de uso validados no Nexus (2024-2026)

- **Cursos online:** 1.247 afiliados lançaram sua plataforma com Lovable.
- **Ferramentas internas:** dashboards de equipe, CRM leve, tracker de afiliado.
- **Comunidades pagas:** com área de membros, chat, gamificação.
- **Apps de produtividade:** geradores de copy, agendadores, calculators.
- **MVPs de SaaS:** validação de ideia antes de levantar investimento.

1.4 — Comparativo: Lovable vs alternativas

Plataforma	Foco	Limite prático
Lovable	Apps full-stack com backend	Excelente até 50k MAU
Bolt.new	Apps web rápidos	Backend limitado
v0.dev	UI components React	Apenas frontend
Replit Agent	Apps completos + IDE	Mais lento, mais flexível
Cursor Composer	IDE agentic para devs	Requer conhecimento técnico

1.5 — Quando NÃO usar Lovable

- Apps mobile nativos (use React Native + Expo direto).
- Sistemas com requisitos regulatórios pesados (bancos, saúde).
- Projetos com > 1M MAU (requer arquitetura custom).
- Produtos que dependem de WebGL/canvas pesado.
- Situações onde dados não podem sair de datacenter brasileiro (use on-prem).

Regra prática Nexus: se sua ideia de SaaS pode ser um "Notion + Stripe + Chat" com regras de negócio, ela cabe no Lovable. Se precisa de renderização 3D ou processamento de mídia pesado, vai para outra stack.

2

Arquitetura técnica do Lovable

Por dentro, o Lovable é um sistema multi-agente orquestrado em torno de um LLM central (Claude Sonnet 4.5) com ferramentas específicas para cada fase do desenvolvimento. Vamos dissecar.

2.1 — Os 7 agentes especializados

1. **Product Agent:** traduz sua ideia em PRD estruturado (User Stories, Critérios de Aceitação, Edge Cases).
2. **Architect Agent:** escolhe stack, define schema do banco, mapeia APIs externas.
3. **Frontend Agent:** gera componentes React + Tailwind + shadcn/ui.
4. **Backend Agent:** implementa Edge Functions, RLS policies, triggers no Supabase.
5. **Database Agent:** escreve migrations SQL, otimiza índices, valida integridade.
6. **Test Agent:** gera testes unitários e E2E com Playwright.
7. **Deploy Agent:** configura Vercel, secrets, domínio custom, monitoring.

2.2 — Fluxo de raciocínio interno

Quando você envia "crie um app de lista de tarefas com login", o que acontece em ~14 segundos:

```
PROMPT → Product Agent (PRD)
      ↓
      Architect Agent (escolha stack: React+Vite+Supabase)
      ↓
      Schema: profiles, tasks, sessions (RLS)
      ↓
```

```
Frontend Agent (componentes: TaskList, TaskItem, AuthForm)
↓
Backend Agent (Edge Functions: createTask, toggleTask)
↓
Test Agent (smoke tests críticos)
↓
Preview URL gerada
↓
PR aberto no GitHub
```

2.3 — Memória de longo prazo (RAG sobre seu repo)

O Lovable mantém um índice vetorial do seu repositório. Cada nova instrução é comparada semanticamente com arquivos existentes, evitando duplicar código e mantendo a coerência arquitetural.

2.4 — Limites técnicos do Lovable v3 (junho 2026)

Recurso	Limite	Workaround
Tamanho do projeto	~200k LOC	Refatorar periodicamente
Edge Functions simultâneas	500/ms (plano Pro)	Cache + rate limiting
Banco Supabase	500 MB (free) / 8 GB (Pro)	Particionar tabelas grandes
Bandwidth Vercel	100 GB/mês (Pro)	CDN, otimizar imagens
Mensagens/mês	100 (free) / ilimitado (Business)	Modo "edição direta" para tasks pequenas

2.5 — Por que isso é melhor que contratar uma agência

Comparativo de ROI — projeto de 30 dias

Agência tradicional: R\$ 25.000-80.000, 30-60 dias, 3-5 profissionais, retrabalho caro.

Dev freelancer: R\$ 8.000-20.000, 20-40 dias, 1 pessoa, gargalo de comunicação.

Lovable + operador Nexus: R\$ 500-2.000 (subscription), 3-10 dias, 1 PM-Engineer.

Para produtos com escopo < 200 telas, a equação é imbatível. Acima disso, avalie híbrido (Lovable para prototipagem + time para hardening).

2.6 — Estrutura típica de um projeto Lovable

```
meu-app/  
├─ src/  
│  ├─ components/  
│  │  ├─ ui/           # shadcn primitives  
│  │  └─ features/     # TaskList, TaskItem  
│  ├─ pages/           # rotas (React Router)  
│  ├─ hooks/           # useAuth, useTasks  
│  ├─ integrations/  
│  │  └─ supabase/     # client gerado  
│  └─ lib/             # utils, validators  
├─ supabase/  
│  ├─ migrations/      # SQL versionado  
│  ├─ functions/       # Edge Functions (Deno)  
│  └─ config.toml  
├─ public/             # assets estáticos  
├─ tests/              # Playwright E2E  
└─ .env                # secrets (não comitar)
```

2.7 — O que o Lovable NÃO é

- Não é um IDE tradicional — você não edita código diretamente (mas pode baixar o repo).
- Não é um CMS — não substitui WordPress para blogs.
- Não é um sistema de templates — cada app é único.
- Não é uma plataforma low-code para usuários não-técnicos (ainda exige vocabulário de produto).

Saber onde o Martelo não bate é tão importante quanto saber onde bate.

3

Do prompt ao PRD executável

O prompt é o novo briefing. A qualidade do app que sai do Lovable é diretamente proporcional à qualidade do prompt que entra. Esta seção ensina a escrever prompts que viram PRDs.

3.1 — A fórmula P.E.R. (Persona + Escopo + Restrição)

- **Persona:** quem vai usar? (ex: "profissionais de marketing digital, 25-45 anos, nível técnico intermediário")
- **Escopo:** o que precisa fazer no MVP? (3-5 funcionalidades-chave, não 20)
- **Restrições:** limites de stack, orçamento, prazo, integrações obrigatórias.

Exemplo ruim: "Faça um app de gestão para mim."

Exemplo bom (P.E.R.):

"Crie um app de gestão de tarefas para freelancers de design, com login por email, criação de projetos, tasks por projeto com prazo, dashboard de horas trabalhadas, e integração com Stripe para receber pagamento mensal. Use português brasileiro, layout minimalista em tons de roxo, mobile-first."

3.2 — O método "5 camadas" para prompts complexos

1. **Camada 1 — Contexto do negócio:** "Sou afiliado do programa X, minha audiência é Y..."

2. **Camada 2 — Funcionalidades core:** liste 3-5 features, sem ambiguidade.
3. **Camada 3 — User flow:** descreva 1-2 jornadas críticas (signup, primeiro uso).
4. **Camada 4 — Design system:** paleta, tipografia, referências visuais.
5. **Camada 5 — Critérios de aceite:** "deve funcionar offline", "deve ser acessível WCAG AA", etc.

3.3 — Refinamento iterativo (a alma do processo)

O primeiro output do Lovable raramente é o final. O fluxo de iteração:

1. **Briefing inicial** (5 camadas) → app v0.1 funcional.
2. **Teste de fluxo** (você usa como se fosse usuário real).
3. **Lista de melhorias** priorizada (top 5).
4. **Refinamento incremental** ("adicione filtro por data no dashboard", "mude o botão X para verde", etc.)
5. **Validação** com 3-5 usuários beta antes de escalar.

Truque Nexus: use o modo "edição direta" (botão "Edit code") para mudanças pontuais que você sabe fazer — economiza tokens e tempo.

3.4 — Prompts poderosos para tarefas específicas

```
// Adicionar feature
"Adicione ao app:
- Tabela 'subscriptions' com stripe_customer_id, plan, status
- Edge Function 'create-checkout' que recebe planId e retorna URL Stripe
- Página /billing que lista assinatura atual e botão de upgrade
- Webhook /webhooks/stripe que atualiza status da assinatura
- Email de boas-vindas via Resend após primeiro pagamento"

// Refatorar
"Esse componente TaskList tem 380 linhas e ficou difícil de manter.
Quebre em 3 componentes: TaskListHeader, TaskFilters, TaskGrid.
Mantenha o comportamento idêntico, mas adicione testes unitários."

// Debugar
"O botão 'Salvar' não está persistindo no banco. Verifique:
1. Se a Edge Function está sendo chamada (network tab)
2. Se o RLS policy permite insert para o usuário atual"
```

3. Se o estado React está sendo resetado após submit
Mostre o código corrigido."

x

3.5 — Anti-padrões de prompt

- ^x "Faça um app igual ao Airbnb" — cópia literal é inviável e antiético.
- ^x "Adicione tudo que for legal" — leva a feature bloat e débito técnico.
- ^x "Use a melhor tecnologia" — sem contexto, ele escolhe errado.
- "Mude tudo de novo" — perde o histórico de decisões.
- Prompts > 800 palavras — dilui o sinal.

3.6 — Gerenciamento de contexto longo

Em projetos grandes, o Lovable tem um "resumo automático" que comprime o histórico. Para preservar decisões importantes, use o AGENTS.md na raiz do projeto:

```
# AGENTS.md
## Decisões arquiteturais
- Usamos Supabase para auth + DB (decidido em 2026-05-12)
- Stripe para billing (decidido em 2026-05-15)
- shadcn/ui + Tailwind para UI (decidido em 2026-05-20)

## Convenções
- Nomes de tabela no plural (tasks, users)
- Sempre usar RLS em todas as tabelas
- Testes E2E para fluxos críticos: signup, checkout, dashboard
- Não usar emojis no código (apenas em UI)
```

Esse arquivo vira "memória institucional" do seu projeto.

4 Stack automática e customização

O Lovable gera automaticamente um stack moderno e battle-tested. Vamos detalhar cada peça.

4.1 — Frontend

- **React 18** com Server Components quando aplicável
- **Vite** para build (HMR em < 50ms)
- **TypeScript** estrito (zero any no template)
- **Tailwind CSS** + **shadcn/ui** (componentes copy-paste)
- **React Router 6** para SPA routing
- **TanStack Query** para data fetching/cache
- **React Hook Form** + **Zod** para formulários e validação

4.2 — Backend

- **Supabase** (PostgreSQL gerenciado) — auth, DB, storage, realtime, edge functions
- **Edge Functions** em Deno (TypeScript) para lógica de servidor
- **Row Level Security (RLS)** ativado por padrão em todas as tabelas
- **pg_cron** para jobs agendados (cleanup, relatórios)

4.3 — Deploy & Hosting

- **Vercel** (frontend) — edge network global, ISR, preview deploys por PR
- **Supabase Cloud** (backend) — multi-region, backups diários
- **Domínio custom** em 1 clique (configuração DNS automática)
- **SSL/TLS** automático via Let's Encrypt

Custo médio mensal (app 1k MAU):

- Lovable Business: R\$ 350
- Vercel Pro: R\$ 100
- Supabase Pro: R\$ 100
- Domínio: R\$ 5/mês (registro.br)
- **Total: ~R\$ 555/mês** para um SaaS com 1.000 usuários ativos.

4.4 — Customizando a stack: quando e como

Em alguns casos, a stack automática não é a ideal. Você pode customizar:

4.4.1 — Trocar autenticação

Pode usar Clerk, Auth0 ou Keycloak no lugar do Supabase Auth. Custo: precisa configurar OAuth, webhooks, migrations de usuários. Faça isso se:

- Já tem usuários em outro sistema (SSO/Migração).
- Precisa de features avançadas (MFA, passwordless custom).
- Compliance exige IdP específico (ex: gov.br para SaaS gov).

4.4.2 — Trocar banco de dados

Alternativas ao Supabase: Neon, PlanetScale, Railway. Útil se:

- Precisa de read replicas geográficas.
- Volume > 100 GB.
- Requisitos específicos (PostGIS, TimescaleDB).

4.4.3 — Adicionar serviços externos

O Lovable suporta integração nativa com:

- **Stripe** (pagamentos)
- **Resend** (emails transacionais)
- **OpenAI/Anthropic** (AI features)
- **Cloudflare R2** (storage)
- **Sentry** (error tracking)
- **Posthog** (analytics + feature flags)

Mantenha simples: cada dependência externa adiciona risco, custo e latência. Adicione apenas quando o Supabase nativo não resolver.

4.5 — Como customizar via prompt

```
// Trocar de Supabase Auth para Clerk
"Migre a autenticação de Supabase Auth para Clerk:
- Adicione @clerk/clerk-react
- Configure CLERK_PUBLISHABLE_KEY no .env
- Substitua useAuth() por useUser() do Clerk
- Mantenha RLS no Supabase, mas use clerk_user_id em vez de auth.uid()
- Atualize todas as páginas e componentes que usam auth"

// Adicionar feature de IA
"Adicione um campo 'ai_summary' na tabela 'projects' do tipo text.
Crie uma Edge Function 'generate-summary' que:
1. Recebe projectId
2. Busca todas as tasks do projeto
3. Chama Claude API (ANTHROPIC_API_KEY no .env)
4. Salva o resumo na coluna ai_summary
Adicione um botão 'Gerar resumo' no dashboard do projeto."
```

4.6 — Versionamento e migrações

Cada mudança do Lovable gera um commit Git. O fluxo:

1. Mensagem enviada → Lovable gera diff → PR aberto no GitHub.
2. Você revisa (ou aprova automaticamente, se confia).
3. Merge → deploy automático em preview.
4. Após testes, merge em main → produção.

Para bancos, as migrations são versionadas em `supabase/migrations/` e aplicadas automaticamente em ordem cronológica. Nunca edite uma migration já aplicada — crie uma nova.

5

Multi-tenancy, auth e RBAC

Se você quer vender seu SaaS para múltiplos clientes (B2B, áreas de membros, afiliados), precisa lidar com **multi-tenancy** — o isolamento de dados entre organizações.

5.1 — Três padrões arquiteturais

Padrão	Complexidade	Quando usar
Schema por tenant	Alta	Empresas grandes, requisitos regulatórios
Database por tenant	Média	Plano Enterprise com SLA dedicado
Row-Level (shared schema)	Baixa	SaaS B2B padrão (recomendado)

O Lovable adota o padrão **Row-Level** por padrão: uma única tabela, com coluna `tenant_id` e RLS policies que filtram por organização.

5.2 — Implementando multi-tenancy no Lovable

```
// Tabela tenants
CREATE TABLE tenants (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  slug text UNIQUE NOT NULL,
  created_at timestamptz DEFAULT now()
);

// Adicionar tenant_id em todas as tabelas
ALTER TABLE tasks ADD COLUMN tenant_id uuid REFERENCES tenants(id);

// RLS policy
CREATE POLICY "tenant_isolation" ON tasks
```

```
FOR ALL USING (tenant_id = (auth.jwt() ->> 'tenant_id')::uuid);

// Custom claim no JWT
CREATE FUNCTION set_tenant_claim()
RETURNS trigger AS $$
BEGIN
    PERFORM set_config('request.jwt.claims',
        json_build_object('tenant_id', NEW.tenant_id)::text, true);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

5.3 — Autenticação e RBAC

O Supabase Auth suporta email/senha, OAuth (Google, GitHub, etc.) e magic links. Para RBAC (Role-Based Access Control):

1. Crie tabela `roles` com roles disponíveis (owner, admin, member, viewer).
2. Crie tabela `user_roles` (user_id, tenant_id, role).
3. Adicione RLS policies que checam role antes de permitir ação.

```
// Função helper para checar role
CREATE FUNCTION has_role(required_role text)
RETURNS boolean AS $$
  SELECT EXISTS (
    SELECT 1 FROM user_roles
    WHERE user_id = auth.uid()
      AND tenant_id = (auth.jwt() ->> 'tenant_id')::uuid
      AND role = required_role
  );
$$ LANGUAGE sql STABLE;

// Policy: só admins podem deletar
CREATE POLICY "admin_can_delete" ON tasks
  FOR DELETE USING (has_role('admin') OR has_role('owner'));
```

5.4 — Convite de novos membros

Fluxo padrão:

1. Owner/admin digita email do novo membro.
2. Edge Function `invite-member` envia email via Resend.
3. Link contém token JWT de uso único.
4. Usuário clica, cria senha, é adicionado ao tenant com role especificado.

Detalhe crítico: nunca coloque o user_id na URL pública.
Use sempre token opaco (UUID v4 com expiração).

5.5 — Segurança em multi-tenancy

- **SEMPRE ative RLS** em qualquer tabela com dados de usuário.
- **SEMPRE teste cross-tenant access:** faça um teste E2E onde usuário de tenant A tenta acessar dados de tenant B — deve falhar.
- **Nunca exponha IDs internos** em URLs públicas (use slugs ou UUIDs opacos).
- **Logs de auditoria:** registre toda ação sensível (delete, role change, export).
- **Rate limiting** por tenant (não por IP) — use Upstash ou middleware custom.

5.6 — Testando isolamento

Para cada tabela multi-tenant, crie testes Playwright que validam:

```
test('user from tenant A cannot read tenant B tasks', async ({ page }) => {
  // Login como user A
  await loginAs(page, 'a@nexusa.com');
  await page.goto('/tasks');
  // Tentar acessar task de tenant B via API direta
  const response = await page.request.get('/api/tasks?tenantId=B');
  expect(response.status()).toBe(403);
  // Tentar ver na UI
  await page.goto('/tasks?tenantId=B');
  await expect(page.locator('text=Access denied')).toBeVisible();
});
```

Esse tipo de teste pega 90% dos bugs de multi-tenancy antes de ir para produção.

6

Billing, Stripe e SaaS recorrente

Para monetizar seu app Lovable, você precisa de um sistema de billing confiável. O padrão da indústria é Stripe, e o Lovable tem integração nativa.

6.1 — Setup inicial

1. Crie conta Stripe (kyc + 2 dias para ativar).
2. Configure produtos no Stripe Dashboard: Starter (R\$ 49), Pro (R\$ 149), Enterprise (custom).
3. Adicione STRIPE_SECRET_KEY e STRIPE_WEBHOOK_SECRET no .env do Lovable.
4. Crie Edge Function create-checkout que retorna URL Stripe.

```
// Edge Function: create-checkout
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import Stripe from "https://esm.sh/stripe@13.0.0?target=denonext";

const stripe = new Stripe(Deno.env.get("STRIPE_SECRET_KEY")!, {
  apiVersion: "2024-06-20",
});

serve(async (req) => {
  const { priceId, tenantId } = await req.json();
  const session = await stripe.checkout.sessions.create({
    mode: "subscription",
    line_items: [{ price: priceId, quantity: 1 }],
    success_url: `${req.headers.get("origin")}/billing/success`,
    cancel_url: `${req.headers.get("origin")}/billing`,
    metadata: { tenant_id: tenantId },
  });
  return new Response(JSON.stringify({ url: session.url }), {
    headers: { "Content-Type": "application/json" },
  });
});
```

```
});  
});
```

6.2 — Webhooks do Stripe

Eventos críticos para processar:

Evento	Ação
checkout.session.completed	Criar/atualizar subscription, enviar email de boas-vindas
customer.subscription.updated	Atualizar status (active, past_due, canceled)
customer.subscription.deleted	Marcar conta como free, enviar email de re-engajamento
invoice.payment_failed	Notificar usuário, bloquear features após 3 tentativas

```
// Webhook handler
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import Stripe from "https://esm.sh/stripe@13.0.0";

const stripe = new Stripe(Deno.env.get("STRIPE_SECRET_KEY")!);
const endpointSecret = Deno.env.get("STRIPE_WEBHOOK_SECRET")!;

serve(async (req) => {
  const sig = req.headers.get("stripe-signature")!;
  const body = await req.text();
  let event;
  try {
    event = stripe.webhooks.constructEvent(body, sig, endpointSecret);
  } catch (err) {
    return new Response(`Webhook Error: ${err.message}`, { status: 400 });
  }

  if (event.type === "checkout.session.completed") {
    const session = event.data.object;
    const tenantId = session.metadata.tenant_id;
    await supabase.from("subscriptions").upsert({
```

```
        tenant_id: tenantId,  
        stripe_customer_id: session.customer,  
        stripe_subscription_id: session.subscription,  
        status: "active",  
    });  
}  
return new Response("ok", { status: 200 });  
});
```

6.3 — Customer Portal do Stripe

Em vez de construir UI de billing do zero, redirecione para o portal gerenciado do Stripe:

```
// Edge Function: customer-portal
const session = await stripe.billingPortal.sessions.create({
  customer: customerId,
  return_url: `${origin}/billing`,
});
return new Response(JSON.stringify({ url: session.url }));
```

Com isso, seu usuário pode atualizar cartão, ver faturas, cancelar — tudo gerenciado pelo Stripe (PCI-DSS compliant por padrão).

6.4 — Feature gating com base no plano

Para limitar features por plano:

```
// Hook usePlan
export function usePlan() {
  const { data: subscription } = useQuery({
    queryKey: ['subscription'],
    queryFn: () => supabase.from('subscriptions').select('*').single()
  });

  const hasFeature = (feature: string) => {
    const planFeatures = {
      starter: ['tasks', 'projects'],
      pro: ['tasks', 'projects', 'ai_summary', 'integrations'],
      enterprise: ['*'] // todos
    };
    const plan = subscription?.plan || 'starter';
    return planFeatures[plan].includes('*') ||
      planFeatures[plan].includes(feature);
  };

  return { subscription, hasFeature };
}
```

```
// Uso no componente
const { hasFeature } = usePlan();
{hasFeature('ai_summary') && }
```

6.5 — Métricas financeiras do SaaS

Para um SaaS B2B, as métricas que importam são:

- **MRR (Monthly Recurring Revenue):** soma das assinaturas ativas mensais.
- **ARR (Annual Recurring Revenue):** $MRR \times 12$.
- **Churn rate:** % de clientes que cancelam por mês.
- **LTV (Lifetime Value):** receita média por cliente até cancelar.
- **CAC (Customer Acquisition Cost):** quanto custa adquirir 1 cliente.
- **LTV/CAC ratio:** saudável é > 3 .

Use o Stripe Sigma (SQL nativo) ou integre com Metabase/Posthog para visualizar.

7 Integrações com APIs externas

O poder de um SaaS Nexus não está no que ele faz sozinho, mas em como ele conversa com o resto do mundo. Esta seção cobre os padrões de integração.

7.1 — Webhooks: receber eventos

Para receber eventos de sistemas externos (Hotmart, Kiwify, ActiveCampaign, etc.), exponha endpoints HTTP:

```
// Edge Function: webhooks/hotmart
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js";

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
);

serve(async (req) => {
  // Validar assinatura do Hotmart (HMAC-SHA256)
  const signature = req.headers.get("x-hotmart-signature")!;
  const hmacSecret = Deno.env.get("HOTMART_WEBHOOK_SECRET")!;
  const body = await req.text();
  const isValid = await verifyHmac(body, signature, hmacSecret);
  if (!isValid) return new Response("Invalid signature", { status: 400 });

  const event = JSON.parse(body);

  // Processar evento de compra aprovada
  if (event.event === "PURCHASE_APPROVED") {
    const { email, product } = event.data.buyer;
    await supabase.from("users").upsert({
      email,
      product_id: product.id,
    });
  }
});
```

```
        status: "active",
    }, { onConflict: "email" });

    // Enviar email de boas-vindas
    await fetch(`${Deno.env.get("SUPABASE_URL")}/functions/v1/send-welcome-email`, {
        method: "POST",
        body: JSON.stringify({ email, product }),
    });
}

return new Response("ok", { status: 200 });
});
```

7.2 — API Keys e autenticação servidor-a-servidor

Para chamar APIs externas (OpenAI, Anthropic, Resend, etc.):

1. Guarde chaves como secrets no Supabase (nunca no código).
2. Use sempre a chave de menor privilégio (read-only se possível).
3. Rotacione chaves a cada 90 dias.
4. Monitore uso anômalo (alerta se custo diário > 3× média).

```
// Padrão para chamar LLM
const response = await fetch("https://api.anthropic.com/v1/messages",
  method: "POST",
  headers: {
    "x-api-key": Deno.env.get("ANTHROPIC_API_KEY")!,
    "anthropic-version": "2023-06-01",
    "content-type": "application/json",
  },
  body: JSON.stringify({
    model: "claude-sonnet-4-5",
    max_tokens: 1024,
    messages: [{ role: "user", content: prompt }],
  }),
);
const { content } = await response.json();
```

7.3 — Integrações mais comuns no ecossistema Nexus

Serviço	Uso típico	Complexidade
Hotmart / Kiwify	Webhooks de venda	Baixa
ActiveCampaign / RD	Email marketing, automação	Média

WhatsApp Business API	Notificações transacionais	Média
OpenAI / Anthropic	AI features no produto	Alta (custo)
Zapier / Make	Orquestração no-code	Baixa

7.4 — Filas e processamento assíncrono

Para tasks longas (geração de vídeo, processamento de imagem, exports PDF), use filas:

- **Supabase pg_cron** para jobs simples agendados.
- **Inngest / Trigger.dev** para workflows complexos com retries.
- **Edge Function + Realtime** para progresso em tempo real.

```
// Padrão: gerar relatório e notificar usuário
// 1. Edge Function cria job
const { data: job } = await supabase.from("jobs").insert({
  type: "generate_report",
  status: "pending",
  tenant_id,
  payload: { month: "2026-05" },
}).select().single();

// 2. Retorna imediatamente
return new Response(JSON.stringify({ job_id: job.id }), { status: 202 });

// 3. Inngest processa em background
inngest.createFunction(
  { id: "generate-report" },
  { event: "job/created" },
  async ({ event, step }) => {
    const report = await step.run("generate", async () => {
      return await generatePDFReport(event.data.payload);
    });
    await step.run("notify", async () => {
      await sendEmail(event.data.tenant_id, "Relatório pronto", report);
    });
  }
);
```

7.5 — Resiliência e retries

Toda chamada externa pode falhar. Padrões de resiliência:

- **Retry com backoff exponencial:** 1s, 2s, 4s, 8s (até 5 tentativas).
- **Circuit breaker:** após 5 falhas, parar de chamar por 5 minutos.
- **Idempotência:** use Idempotency-Key em POSTs críticos.
- **Dead letter queue:** eventos que falharam 3x vão para revisão manual.

8 Debugging, logs e observabilidade

Em produção, "funciona na minha máquina" não paga as contas. Você precisa de visibilidade.

8.1 — As 3 camadas de observabilidade

1. **Logs:** o que aconteceu (eventos discretos).
2. **Métricas:** quanto e com que frequência (agregados numéricos).
3. **Traces:** caminho da requisição (latência por hop).

8.2 — Ferramentas recomendadas

Camada	Ferramenta	Custo
Logs	Logflare (built-in Supabase) + Sentry	R\$ 0-100/mês
Métricas	Vercel Analytics + Posthog	R\$ 0-200/mês
Traces	Sentry Performance	R\$ 0-150/mês
Uptime	Better Uptime / Cronitor	R\$ 30-100/mês

8.3 — Logging estruturado: o padrão Nexus

```
// Helper de log
export function logEvent(event: string, data: Record) {
  console.log(JSON.stringify({
    timestamp: new Date().toISOString(),
    level: "info",
    event,
    tenant_id: data.tenantId,
    user_id: data.userId,
```

```
        duration_ms: data.duration,  
        ...data,  
    }));  
}  
  
// Uso  
logEvent("task_created", {  
    taskId: task.id,  
    tenantId,  
    userId: user.id,  
    source: "edge_function",  
});
```

Com logs JSON estruturados, você consegue buscar, agregar e alertar com facilidade.

8.4 — Debugging comum no Lovable

8.4.1 — RLS policy bloqueando acesso

Sintoma: query retorna vazio mesmo com dados existentes.

Diagnóstico: rode a query como o usuário autenticado no SQL Editor do Supabase. Se retornar vazio, é RLS.

Fix: ajuste a policy para incluir a condição correta.

8.4.2 — Edge Function timeout

Sintoma: 504 Gateway Timeout após 30s.

Diagnóstico: divida a função em partes, meça cada uma com `console.time`.

Fix: use fila assíncrona para `tasks > 10s`, retorne `job_id` imediatamente.

8.4.3 — CORS error em fetch

Sintoma: "No 'Access-Control-Allow-Origin' header" no console.

Fix: adicione headers CORS na Edge Function:

```
return new Response(JSON.stringify(data), {
  headers: {
    "Content-Type": "application/json",
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Headers": "authorization, content-type",
  },
});
```

8.5 — Testes que salvam vidas

- **Smoke tests E2E:** signup → onboarding → primeira ação core (Playwright).
- **Testes de billing:** use Stripe CLI + test mode para simular webhooks.

- **Testes de carga:** k6 ou Artillery para validar que aguenta 1k usuários simultâneos.
- **Testes de segurança:** OWASP ZAP scan mensal.

Regra 80/20: 80% dos bugs vêm de 20% dos fluxos. Teste exaustivamente: signup, checkout, primeira ação. O resto pode ter cobertura mínima.

9 Governança, segurança e escala

Construir um MVP é uma coisa. Manter um SaaS em produção com clientes pagantes é outra. Esta seção cobre os processos que mantêm seu produto saudável.

9.1 — Segurança essencial para SaaS Nexus

- ✓ HTTPS obrigatório (Vercel + Supabase já garantem).
- ✓ Headers de segurança: CSP, HSTS, X-Frame-Options, X-Content-Type-Options.
- ✓ Autenticação com 2FA para usuários sensíveis (admins).
- ✓ Senhas hasheadas (Supabase Auth já faz com bcrypt).
- ✓ Sessões curtas + refresh tokens rotativos.
- ✓ Rate limiting em APIs públicas.
- ✓ Logs de auditoria para ações sensíveis.
- ✓ Backups diários do banco (Supabase Pro já faz).
- ✓ Plano de resposta a incidentes documentado.

9.2 — LGPD para SaaS B2B

Seus clientes (empresas) são co-controladores, mas você (operador) tem obrigações:

1. **DPO:** indique um (pode ser você mesmo se for < 10k titulares).
2. **Política de privacidade:** explique quais dados coleta, por quê, com quem compartilha.

3. **Direitos do titular:** implemente endpoints para export, correção e exclusão de dados.
4. **RIPD:** Relatório de Impacto à Proteção de Dados Pessoais para tratamento de alto risco.
5. **Encarregado:** canal de contato claro (email DPO).

Use o template Nexus de política (disponível na Academia) como ponto de partida.

9.3 — Escala: do MVP ao enterprise

O caminho típico de um SaaS Lovable:

Fase	Usuários	Receita	Desafios
0 → 1 (MVP)	0 - 100	R\$ 0 - 5k/mês	Validar product-market fit
1 → 10 (Growth)	100 - 1k	R\$ 5k - 50k/mês	Onboarding, churn, suporte
10 → 100 (Scale)	1k - 10k	R\$ 50k - 500k/mês	Performance, multi-region, RBAC avançado
100 → 1000 (Enterprise)	10k+	R\$ 500k+/mês	SOC2, SLA 99.9%, integrações enterprise

9.4 — Quando migrar para fora do Lovable

Sinais de que você superou o Lovable:

- Mais de 200k LOC.
- Latência P95 > 800ms consistentemente.
- Equipe técnica > 5 devs (colaboração no Lovable é limitada).
- Requisitos regulatórios (HIPAA, PCI-DSS nível 1).
- Customizações deep no compilador (ex: Rust + WebAssembly).

A boa notícia: o Lovable gera código padrão React/Supabase, então migrar para Next.js + AWS é factível em 1-2 sprints com ajuda de uma agência.

9.5 — Comunidade Nexus

Mais de 12.000 afiliados Nexus já lançaram SaaS com Lovable. A comunidade no Discord e nos canais oficiais:

- #showcase: compartilhe seu app e receba feedback.
- #help-desk: suporte peer-to-peer.
- #office-hours: sessão semanal com PM-Engineers experts.
- #playbooks: 200+ templates de prompts e AGENTS.md prontos.

Participe. O próximo nível do seu SaaS está em quem já passou por ele.

10

Glossário técnico e exercícios práticos

Glossário

PRD

Product Requirements Document — especificação de produto.

RLS

Row Level Security — política de acesso por linha no Postgres.

Edge Function

Função serverless executada na borda (Deno).

Multi-tenancy

Arquitetura onde vários clientes compartilham uma instância.

RBAC

Role-Based Access Control — controle de acesso por papel.

SSO

Single Sign-On — login único entre sistemas.

Webhook

Callback HTTP para receber eventos externos.

Idempotency Key

Identificador único para evitar duplicação.

MRR

Monthly Recurring Revenue — receita recorrente mensal.

Churn

Taxa de cancelamento de clientes.

CAC

Customer Acquisition Cost — custo de aquisição.

LTV

Lifetime Value — valor vitalício do cliente.

shadcn/ui

Biblioteca de componentes React copy-paste.

Zod

Biblioteca de validação de schemas TypeScript.

PG_cron

Agendador de jobs nativo do PostgreSQL.

Exercícios práticos

Exercício 1 — Briefing para PRD

Escolha uma ideia de SaaS que você sempre quis lançar. Aplique o método P.E.R. e o template de 5 camadas. Rode no Lovable e avalie o output.

Exercício 2 — Multi-tenancy real

Adicione a tabela tenants e RLS em uma tabela existente. Crie dois tenants de teste, valide que usuário A não vê dados do tenant B.

Exercício 3 — Billing completo

Configure Stripe em modo test. Crie 2 planos, integre checkout e webhook. Simule 5 cenários: sucesso, falha, cancelamento, upgrade, downgrade. Documente os fluxos.

Exercício 4 — Observabilidade

Adicione Sentry + Posthog. Gere 3 tipos de erro proposital (console.error, throw, fetch failed). Valide que aparecem no Sentry com contexto. Crie 1 alerta no Posthog para evento crítico.

Continua no Ebook 03 — Perplexity

Agora que você sabe construir produtos, vamos ensinar a IA a **pesquisar por você**: a arquitetura de RAG avançado, agentes de busca e a nova geração de pesquisa agêntica.

